

# Gift / Points Sharing System — Technical Guide

**Database:** dev\_myunisavedb

**Purpose:** Allow an owner to share wallet balance with recipients based on accepted percentage allocations.

## 1) Overview & Core Concepts

This guide explains the schema, procedures, and workflows to implement a wallet balance sharing feature. Users request a share (as a percentage) from an owner. The owner accepts or rejects these requests. Accepted requests become active allocations. When the owner shares a rupee amount, it is distributed to all active recipients according to their percentage allocations, and recorded in a transfer ledger.

### Key Rules

- Total active allocations for an owner must be  $\leq 100\%$ .
- A request must be accepted before it can become an allocation.
- Distribution rounds each recipient's amount to 2 decimals. Any rounding residue is returned to the owner.
- A recipient may have at most one active allocation per owner.

## 2) Entity Relationships (ER Sketch)

```
users (existing)
    ■■■< user_wallets (1:1 by user_id [recommend UNIQUE])
share_requests (owner_id → users.user_id, requester_id → users.user_id)
    ■■■ share_allocations (owner_id, recipient_id → users.user_id; source_request_id → share_requests.request_id)
transfers (owner_id → users.user_id, recipient_id → users.user_id)
```

Flow:

requester → share\_requests → (owner accepts) → share\_allocations → sp\_share\_owner\_amount → transfers + wallet up

## 3) Table Definitions (Your Current Setup)

### 3.1 user\_wallets

```
CREATE TABLE IF NOT EXISTS user_wallets (
    wallet_id    BIGINT PRIMARY KEY NOT NULL,
    user_id      BIGINT,
    balance_rs   DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

-- Seed example

```
INSERT INTO user_wallets (user_id, balance_rs) VALUES (20, 500.00)
ON DUPLICATE KEY UPDATE balance_rs = VALUES(balance_rs);
```

Recommendation: Enforce one wallet per user and avoid Safe Update errors by making user\_id UNIQUE.

```
ALTER TABLE user_wallets ADD CONSTRAINT ux_wallet_user UNIQUE (user_id);
```

### 3.2 share\_requests

```
CREATE TABLE IF NOT EXISTS share_requests (
    request_id    BIGINT PRIMARY KEY AUTO_INCREMENT,
    owner_id      BIGINT NOT NULL,
    requester_id  BIGINT NOT NULL,
    requested_percent DECIMAL(5,2) NOT NULL,
    status        ENUM('pending','accepted','rejected','cancelled') NOT NULL DEFAULT 'pending',
    created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at  TIMESTAMP NULL DEFAULT NULL,
    CONSTRAINT chk_req_percent CHECK (requested_percent >= 0 AND requested_percent <= 100)
);
```

### 3.3 share\_allocations

```

CREATE TABLE IF NOT EXISTS share_allocations (
  allocation_id      BIGINT PRIMARY KEY AUTO_INCREMENT,
  owner_id           BIGINT NOT NULL,
  recipient_id       BIGINT NOT NULL,
  percent_share       DECIMAL(5,2) NOT NULL,
  source_request_id  BIGINT NULL,
  is_active           TINYINT(1) NOT NULL DEFAULT 1,
  created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_alloc_src FOREIGN KEY (source_request_id) REFERENCES share_requests(request_id),
  CONSTRAINT chk_alloc_percent CHECK (percent_share >= 0 AND percent_share <= 100),
  CONSTRAINT ux_owner_rec UNIQUE (owner_id, recipient_id)
);

-- Note: source_request_id is NULL-able to allow manual allocations if needed.

```

### **3.4 transfers**

```

CREATE TABLE IF NOT EXISTS transfers (
  transfer_id        BIGINT PRIMARY KEY AUTO_INCREMENT,
  owner_id           BIGINT NOT NULL,
  recipient_id       BIGINT NOT NULL,
  amount_rs          DECIMAL(12,2) NOT NULL,
  percent_snapshot   DECIMAL(5,2) NOT NULL,
  created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  note               VARCHAR(255) NULL,
  CONSTRAINT chk_tr_amt CHECK (amount_rs >= 0)
);

```

## 4) Function

```
DROP FUNCTION IF EXISTS fn_total_percent_for_owner;
DELIMITER $$
CREATE FUNCTION fn_total_percent_for_owner(p_owner_id BIGINT)
RETURNS DECIMAL(6,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(6,2);
    SELECT IFNULL(SUM(percent_share), 0.00)
        INTO v_total
    FROM share_allocations
    WHERE owner_id = p_owner_id AND is_active = 1;
    RETURN v_total;
END$$
DELIMITER ;
```

## 5) Procedures

### 5.1 *sp\_send\_share\_request*

```
DROP PROCEDURE IF EXISTS sp_send_share_request;
DELIMITER $$
CREATE PROCEDURE sp_send_share_request(
    IN p_owner_id BIGINT,
    IN p_requester_id BIGINT,
    IN p_requested_percent DECIMAL(5,2)
)
BEGIN
    IF p_owner_id = p_requester_id THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Owner and requester cannot be the same user';
    END IF;
    IF p_requested_percent < 0 OR p_requested_percent > 100 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'requested_percent must be between 0 and 100';
    END IF;

    IF (SELECT COUNT(*) FROM users WHERE user_id = p_owner_id) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Owner does not exist';
    END IF;
    IF (SELECT COUNT(*) FROM users WHERE user_id = p_requester_id) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Requester does not exist';
    END IF;

    IF EXISTS (SELECT 1 FROM share_requests WHERE owner_id = p_owner_id AND requester_id = p_requester_id AND status = 'pending') THEN
        UPDATE share_requests
        SET requested_percent = p_requested_percent
        WHERE owner_id = p_owner_id
        AND requester_id = p_requester_id
        AND status = 'pending';
    ELSE
        INSERT INTO share_requests(owner_id, requester_id, requested_percent, status)
        VALUES (p_owner_id, p_requester_id, p_requested_percent, 'pending');
    END IF;
END$$
DELIMITER ;
```

### 5.2 *sp\_respond\_share\_request*

```
DROP PROCEDURE IF EXISTS sp_respond_share_request;
DELIMITER $$
CREATE PROCEDURE sp_respond_share_request(
    IN p_request_id BIGINT,
    IN p_owner_id BIGINT,
    IN p_accept BOOLEAN
)
BEGIN
    DECLARE v_owner_id BIGINT;
    DECLARE v_requester_id BIGINT;
    DECLARE v_req_percent DECIMAL(5,2);
```

```

DECLARE v_status VARCHAR(16);

START TRANSACTION;

SELECT owner_id, requester_id, requested_percent, status
      INTO v_owner_id, v_requester_id, v_req_percent, v_status
FROM share_requests
WHERE request_id = p_request_id
FOR UPDATE;

IF v_owner_id IS NULL THEN
  ROLLBACK;
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request not found';
END IF;

IF v_status <> 'pending' THEN
  ROLLBACK;
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Request is not pending';
END IF;

IF v_owner_id <> p_owner_id THEN
  ROLLBACK;
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Owner mismatch for this request';
END IF;

IF p_accept THEN
  SET @current_total := fn_total_percent_for_owner(p_owner_id);

  IF EXISTS (SELECT 1 FROM share_allocations WHERE owner_id = p_owner_id AND recipient_id = v_requester_id AND is_active = 1)
    SELECT percent_share INTO @old_pct
    FROM share_allocations
    WHERE owner_id = p_owner_id AND recipient_id = v_requester_id AND is_active = 1
    FOR UPDATE;

    SET @new_total := @current_total - IFNULL(@old_pct,0) + v_req_percent;
  ELSE
    SET @new_total := @current_total + v_req_percent;
  END IF;

  IF @new_total > 100.00 THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Accepting this request would exceed 100% total allocation';
  END IF;

  INSERT INTO share_allocations(owner_id, recipient_id, percent_share, source_request_id, is_active)
  VALUES (p_owner_id, v_requester_id, v_req_percent, p_request_id, 1)
  ON DUPLICATE KEY UPDATE
    percent_share = VALUES(percent_share),
    source_request_id = VALUES(source_request_id),
    is_active = 1;

  UPDATE share_requests
    SET status = 'accepted', responded_at = NOW()
    WHERE request_id = p_request_id;
ELSE
  UPDATE share_requests
    SET status = 'rejected', responded_at = NOW()
    WHERE request_id = p_request_id;
END IF;

COMMIT;
END$$
DELIMITER ;

```

### 5.3 *sp\_share\_owner\_amount*

```

DROP PROCEDURE IF EXISTS sp_share_owner_amount;
DELIMITER $$
CREATE PROCEDURE sp_share_owner_amount(
    IN p_owner_id BIGINT,
    IN p_amount_rs DECIMAL(12,2),
    IN p_note VARCHAR(255)
)
BEGIN
    DECLARE v_balance DECIMAL(12,2);
    DECLARE v_total_pct DECIMAL(6,2);

    DECLARE cur_recipient_id BIGINT;
    DECLARE cur_pct DECIMAL(5,2);
    DECLARE cur_amt DECIMAL(12,2);
    DECLARE done INT DEFAULT 0;
    DECLARE v_distributed DECIMAL(12,2) DEFAULT 0.00;

    DECLARE c CURSOR FOR
        SELECT recipient_id, percent_share
        FROM share_allocations
        WHERE owner_id = p_owner_id AND is_active = 1
        ORDER BY percent_share DESC, recipient_id ASC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    IF p_amount_rs <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Amount must be > 0';
    END IF;

    START TRANSACTION;

    SELECT balance_rs INTO v_balance
    FROM user_wallets
    WHERE user_id = p_owner_id
    FOR UPDATE;

    IF v_balance IS NULL THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Owner wallet not found';
    END IF;

    IF v_balance < p_amount_rs THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient owner balance';
    END IF;

    SET v_total_pct = fn_total_percent_for_owner(p_owner_id);
    IF v_total_pct <= 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No active allocations to share';
    END IF;

    UPDATE user_wallets
        SET balance_rs = balance_rs - p_amount_rs
    WHERE user_id = p_owner_id;

    OPEN c;
    read_loop: LOOP
        FETCH c INTO cur_recipient_id, cur_pct;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        SET cur_amt = ROUND(p_amount_rs * (cur_pct / 100.0), 2);
        SET v_distributed = v_distributed + cur_amt;

        INSERT INTO user_wallets (user_id, balance_rs)
        VALUES (cur_recipient_id, cur_amt)
        ON DUPLICATE KEY UPDATE balance_rs = balance_rs + VALUES(balance_rs);

        INSERT INTO transfers(owner_id, recipient_id, amount_rs, percent_snapshot, note)
        VALUES (p_owner_id, cur_recipient_id, cur_amt, cur_pct, p_note);
    END LOOP;
END

```

```

END LOOP read_loop;
CLOSE c;

IF v_distributed <> p_amount_rs THEN
  UPDATE user_wallets
    SET balance_rs = balance_rs + (p_amount_rs - v_distributed)
  WHERE user_id = p_owner_id;
END IF;

COMMIT;
END$$
DELIMITER ;

```

## 6) Safe Update Mode (Error 1175) — Explanation & Fix

MySQL Workbench's Safe Update Mode blocks UPDATE/DELETE statements that don't filter by a key column. If your update uses a non-indexed column, you'll see Error 1175.

### *Fix Options:*

- Recommended: Add a UNIQUE constraint on user\_wallets.user\_id so WHERE user\_id = ? uses a key:  
`ALTER TABLE user_wallets ADD CONSTRAINT ux_wallet_user UNIQUE (user_id);`
- Or disable Safe Updates temporarily: `SET SQL_SAFE_UPDATES = 0;` -- run your commands `SET SQL_SAFE_UPDATES = 1;` -- re-enable

## 7) End-to-End Workflow Example

```

-- 1) Seed owner wallet
INSERT INTO user_wallets (user_id, balance_rs) VALUES (20, 500.00)
ON DUPLICATE KEY UPDATE balance_rs = VALUES(balance_rs);

-- 2) Requests
CALL sp_send_share_request(20, 21, 10.00);
CALL sp_send_share_request(20, 22, 20.00);
CALL sp_send_share_request(20, 23, 20.00);

-- 3) Owner accepts
CALL sp_respond_share_request(1, 20, TRUE);
CALL sp_respond_share_request(2, 20, TRUE);
CALL sp_respond_share_request(3, 20, TRUE);

-- 4) Confirm allocations total
SELECT fn_total_percent_for_owner(20) AS total_alloc; -- expect 50.00

-- 5) Share amount
CALL sp_share_owner_amount(20, 200.00, 'Monthly share');

-- 6) Verify balances and transfers
SELECT * FROM user_wallets WHERE user_id IN (20,21,22,23);
SELECT * FROM transfers WHERE owner_id = 20 ORDER BY created_at DESC;

```

## 8) Admin Operations

Update an existing allocation (re-validates in respond proc by re-sending a request with new % and accepting it). Alternatively, manual update:

```

-- Manually update percent (ensure total ≤ 100 afterwards)
UPDATE share_allocations SET percent_share = 25.00 WHERE owner_id = 20 AND recipient_id = 21;

-- Deactivate an allocation (soft revoke)
UPDATE share_allocations SET is_active = 0 WHERE owner_id = 20 AND recipient_id = 23;

```

## 9) FAQs

Q: What goes into source\_request\_id? A: The request\_id from share\_requests that led to this allocation. Keep NULL if created manually. Q: Why does Error 1175 appear when running sp\_share\_owner\_amount? A: Safe

Update Mode. Add UNIQUE(user\_id) on user\_wallets or disable Safe Updates temporarily. Q: Where does rounding residue go? A: Back to the owner wallet to ensure exact accounting. Q: Can multiple owners use the system? A: Yes. All logic is keyed by owner\_id and works independently per owner.